

System Design Interview: Design WhatsApp

#106: System Design Interview - Part 1



NEO KIM AND HAYK

DEC 08, 2025  PAID

Get my system design playbook for FREE on
newsletter signup:

✓ Subscribed

- *Share this post & I'll send you some rewards for the referrals.*
- *Block diagrams created using Eraser.*

Designing a real-time chat application like WhatsApp is one of the most common system design interview questions.

In a short period, you need to show that you understand not just the happy path, but also all the things that can go wrong when messages fly between disconnected users on mobile networks.

WhatsApp handles over 100 billion messages daily for 2+ billion users.

We'll build this from scratch, starting with a simple version that works for a few thousand users, then scale it up to handle billions of users sending messages across unreliable mobile networks.

This is a practical guide:

We won't hand-wave away the hard parts or pretend everything works. Instead, we'll build something functional, identify where it breaks, and fix it properly.

What we'll cover in Part 1:

- Requirements and capacity estimation
- Core architecture and data models
- Why WebSockets beat other protocols for real-time messaging
- Service discovery and presence management
- Handling online and offline message delivery
- Multi-device synchronization
- Media file uploads without killing your servers

By the end, you'll understand not just what to build, but why each decision is made and what breaks when you get it wrong.

Onward.

 **Ditch the vibes, get the context (sponsor)**



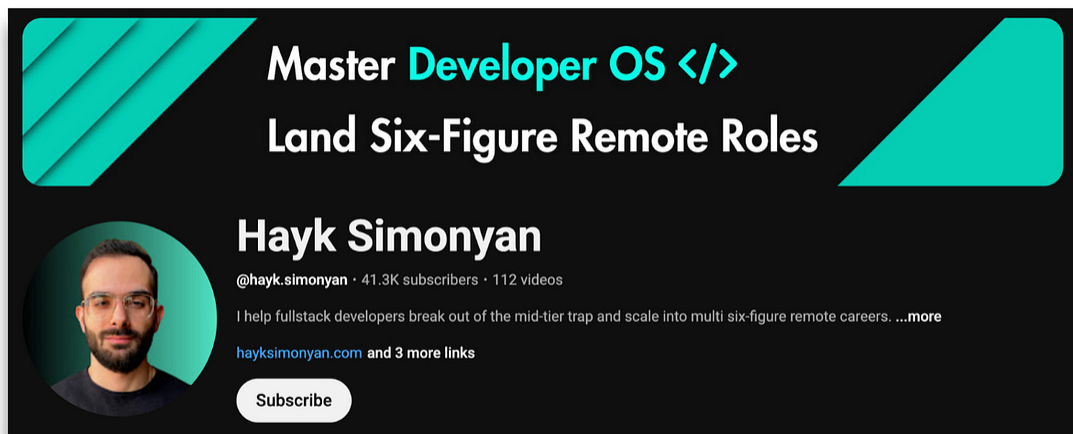
Augment Code's powerful AI coding agent meets professional software developers exactly where they are, delivering production-grade features and deep context into even the gnarliest codebases.

With Augment Code, you can:

- **Keep using VS Code, JetBrains, Android Studio, or even Vim**
- Index and navigate millions of lines of code
- **Get instant answers about any part of your codebase**
- Build with the AI agent that gets you, your team, and your code

👉 **Ditch the Vibes and Get the Context You Need to Engineer What's Next**

I want to introduce Hayk Simonyan as a guest author.



He's a senior software engineer specializing in helping developers break through their career plateaus and secure senior roles.

If you want to master the essential system design skills and land senior developer roles, I highly recommend checking out Hayk's **YouTube channel**.

His approach focuses on what top employers actually care about: system design expertise, advanced project experience, and elite-level interview performance.

Requirements: What Are We Actually Building?

Starting simple... we're designing a real-time messaging app that lets people send text messages and media to each other. But let's get specific about what "real-time" actually means here.

Core requirements:

- Users can send and receive text messages in 1:1 chats
- Group chats with up to 100 participants (we'll see why this number matters later)
- Messages delivered while users are offline get queued up for 30 days
- Media attachments (images, videos, audio clips)
- Message status tracking (sent, delivered, read)
- Online/offline status with *"last seen"*

Non-functional requirements:

- Low latency delivery (under 500ms when users are online)
- Guaranteed message delivery (messages can't just disappear)
- Handle billions of users with high throughput
- Don't store messages on servers longer than necessary
- Stay resilient when individual components fail

The tricky bit most people miss is the offline requirement.

It's easy to design for everyone to be online. But real life doesn't work that way...

Back-of-the-Envelope: Understanding the Scale

Let's run the numbers so we don't build something that collapses under real load.

User metrics:

- 1 billion registered users

- 500 million daily active users
- 50 million concurrent connections during peak hours
- Average 10-20 messages per user daily

Message estimations:

- Daily messages: $500\text{M users} \times 20 \text{ messages} = 10 \text{ billion messages/day}$
- That's roughly 115,000 messages per second on average.
- But during peak hours, we need to multiply that by 3-5x. So we're looking at 350,000-500,000 messages per second.

Storage calculation:

Each message with metadata averages about 1KB (text content, sender/receiver IDs, timestamp, status flags, etc.)

- Daily storage: $1\text{KB} \times 10\text{B messages} = 10 \text{ TB/day}$
- Annual storage: 3.6 PB/year

But here's the thing:

Most messages get delivered and cleared from the server within seconds. We're not storing everything forever. Users download messages to their devices, and we clean them up on the server.

Factor in the 30-day retention for undelivered messages and 10% of users keeping history on servers. And suddenly we're looking at 400-500 TB of active storage instead of petabytes.

Bandwidth:

50M concurrent connections at peak.

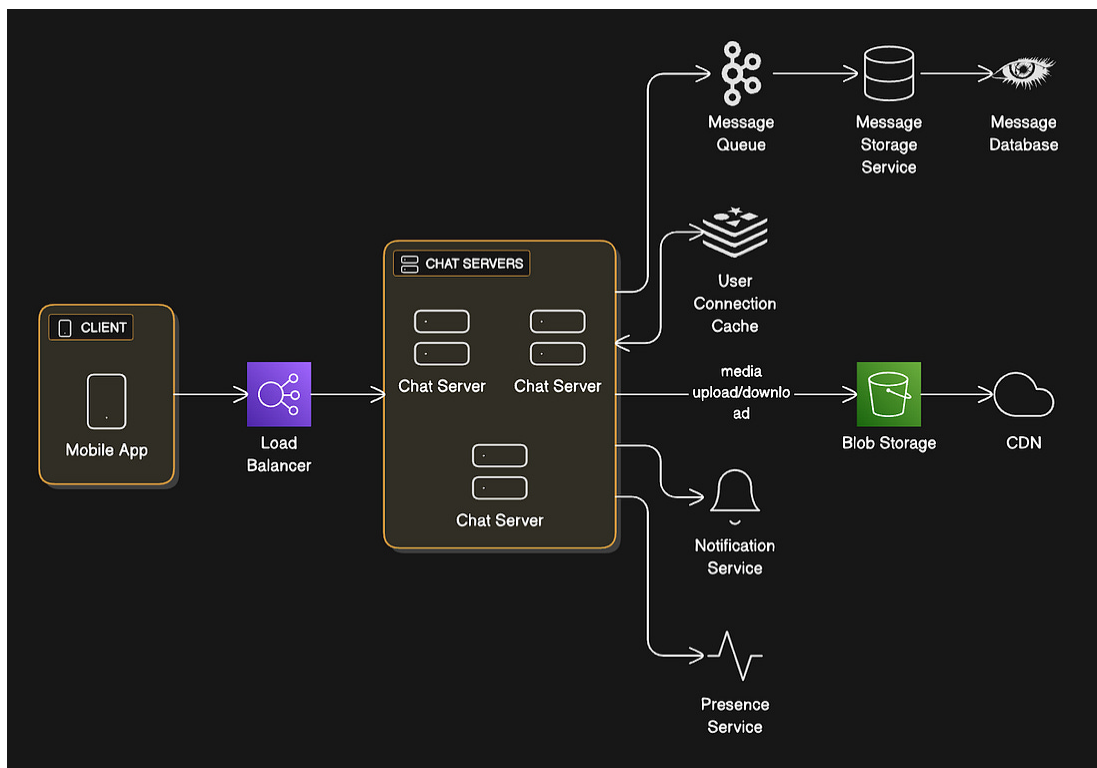
If each connection averages 10KB/s for active messaging, that's 500 GB/s of bandwidth. This is why WhatsApp was famous for running on minimal infrastructure.

They are very efficient in not sending unnecessary data.

High-Level System Design

Let's build this in stages, starting simple and adding complexity only when we need it:

The Basic Components



Mobile App (Client)

The user's phone. It maintains a persistent connection to our backend, handles the UI, manages local message storage, and retries failed operations.

Load Balancer

This sits in front of our chat servers and distributes incoming connections across multiple servers.

Think of it as a traffic cop routing cars to different lanes. It routes incoming WebSocket connections to chat servers and uses sticky sessions (meaning once you connect to Server A, you stay on Server A). Simply put, users don't bounce between servers mid-conversation.

It also monitors server health and stops sending traffic to dead servers.

Chat Servers

Chat servers maintain WebSocket connections to clients (a persistent two-way communication channel that stays open), route messages between users, track who's online, and handle message persistence.

Each server can handle hundreds of thousands of simultaneous connections.

Message Queue

This decouples message writing from delivery.

When a chat server receives a message, it immediately acknowledges receipt to the sender, then asynchronously pushes it to the queue for storage and delivery.

We can use tools like [Kafka](#) or [RabbitMQ](#) that work well here.

Message Storage Service

Consumes from the queue and writes messages to the database.

It also handles querying message history and managing retention policies (such as deleting old messages after 30 days).

Message Database

Persistent storage for messages.

NoSQL works well here ([Cassandra](#), [DynamoDB](#)) because we need high write throughput and our query patterns are straightforward (fetch messages by user/conversation/timestamp).

We're writing billions of messages per day, so we need something that can handle massive write volume without slowing down.

User Connection Cache

In-memory store ([Redis](#)) tracks which users are online, which chat server they're connected to, and their last activity timestamp.

This makes routing decisions fast. Checking Redis takes microseconds, whereas querying a database takes milliseconds. At scale, that difference matters.

Blob Storage + CDN

Blob storage (e.g., [AWS S3](#) or [Google Cloud Storage](#)) stores media files; it holds the actual files.

CDN (Content Delivery Network) caches popular files at edge locations worldwide so downloads are fast regardless of where users are. It offers direct upload/download paths so chat servers don't become bottlenecks for large file transfers.

Notification Service

Handles push notifications for offline users through APNs (Apple Push Notification Service for iOS) and FCM (Firebase Cloud Messaging for Android).

When someone messages you while you're offline, this makes your phone buzz.

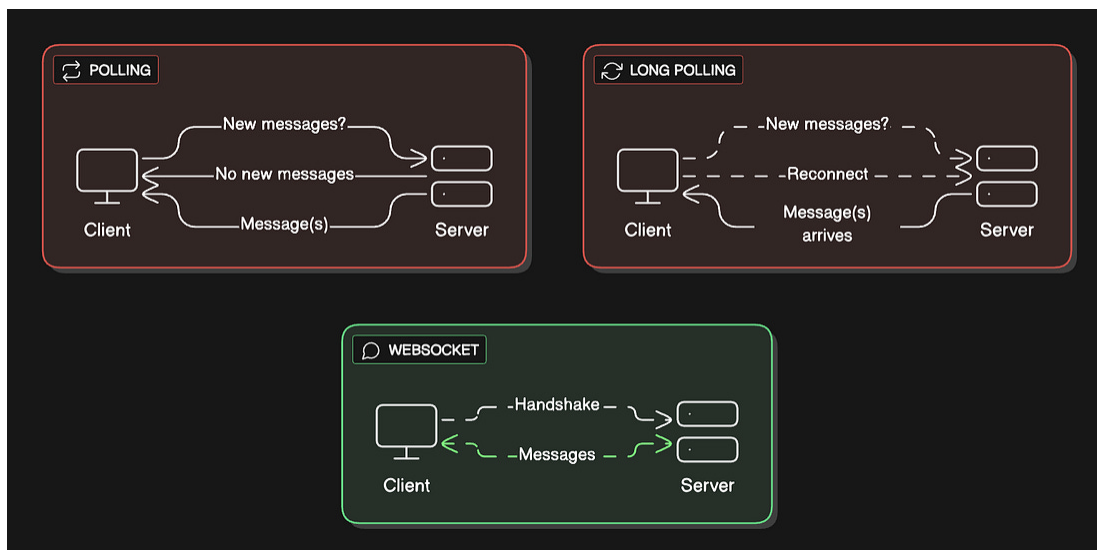
Presence Service

Dedicated service for managing online/offline status. Receives heartbeats from users, updates Redis, and publishes presence changes to interested subscribers.

The WebSocket Decision

Most messaging systems use WebSockets.

Let's understand why by looking at what doesn't work:



Polling

The client asks, “*Any new messages?*” every few seconds.

This wastes bandwidth and adds latency.

If you’re checking every 2 seconds and a message arrives right after you check, you wait 2 seconds to see it. Multiply that by millions of users constantly asking for updates even when there’s nothing new, and you’re burning resources for no reason.

Long polling

Client opens a request, server holds it open until there’s a message or a timeout.

Long polling is better than regular polling.

But it’s still problematic:

You’re constantly opening and closing connections. Each message requires a full HTTP handshake (the back-and-forth to establish a connection).

At scale, this overhead kills you. Your servers spend more time managing connection lifecycles than actually delivering messages. Plus, idle HTTP connections still consume server resources (memory for connection state, thread pool slots) without the efficiency benefits of WebSockets.

WebSockets (WHAT WE USE)

One persistent bidirectional connection that stays open. Both the client and the server can push data at any time. Minimal overhead once established.

Why we chose WebSocket:

The connection remains open throughout the session. When User A sends a message to User B, it flows through A's WebSocket connection to the server, which pushes it through B's WebSocket connection instantly.

No polling and no repeated handshakes. Just data flowing both ways. The connection overhead happens only once when you open the app; everything after that is just message content.

WebSocket Tradeoffs:

- It requires special load balancers that support them (Layer 4 load balancing instead of Layer 7).
- They also use more memory per connection on the server since you're keeping connections open.

But for real-time messaging, the latency and bandwidth benefits make it worthwhile.

Handling Idle WebSocket Connections:

What happens when a user opens WhatsApp but doesn't send any messages for hours?

The connection stays open but idle!

We handle this with heartbeats: Client sends a lightweight ping every 30 seconds. Server responds with Pong.

If the server doesn't receive a heartbeat for 60 seconds, it assumes the connection died (network issue, app backgrounded) and closes it. This prevents the accumulation of zombie connections that waste server resources.

Idle connections still consume memory (connection state, socket buffers), but the cost is minimal compared to the benefit of instant

message delivery when the user sends something.

Identifying Servers That Don't Stand Still

In production, chat servers come and go... they crash, get deployed, scale up and down. So how does the system keep track of what's available?

The Problem:

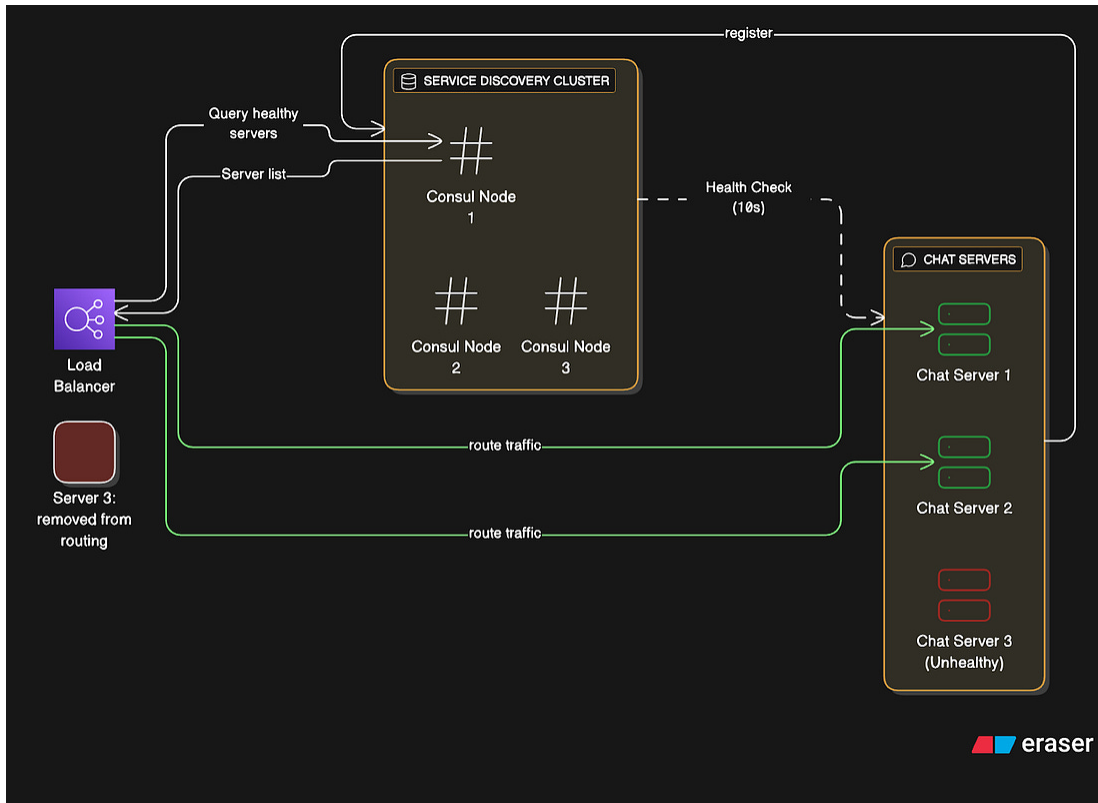
Your load balancer needs to know which chat servers are healthy.

Chat servers need to know which other services are available (Message Storage Service instances, Redis cluster nodes). Hardcoding IPs doesn't work when instances are ephemeral.

Solution: Service Registry Pattern

We use a service registry such as [Consul](#) or [AWS Cloud Map](#).

Here's how it works:



Registration:

When the Chat Server starts:

1. Server boots up on IP 10.0.1.45:8080
2. Registers itself with Consul:
 - Service name: "chat-server"
 - Instance ID: "chat-server-abc123"
 - Health check endpoint: "/health"
 - Metadata: {region: "us-east", capacity: 100000}
3. Sends a heartbeat every 10 seconds
4. If the heartbeat stops, Consul marks it unhealthy after 30 seconds

Discovery:

When the Load Balancer needs available servers:

1. Query Consul: *"Give me all healthy chat-server instances"*
2. Consul returns list: [10.0.1.45:8080, 10.0.1.67:8080, ...]
3. Load balancer updates the routing table
4. Watches for changes (Consul notifies on updates)

Why we chose this:

Services discover each other dynamically.

- Deploy new servers; they auto-register.
- Kill a server, and it's automatically removed.

No manual configuration updates.

Health Checks:

Consul hits this endpoint every 10 seconds.

If the instance fails three times in a row, it is marked unhealthy.

```
Health Checks

app.get('/health', (req, res) => {
  const checks = {
    websocket_capacity: activeConnections < maxConnections,
    redis_connected: redisClient.isReady,
    memory_ok: process.memoryUsage().heapUsed < maxMemory
  };

  const healthy = Object.values(checks).every(v => v);
  res.status(healthy ? 200 : 503).json(checks);
});
```

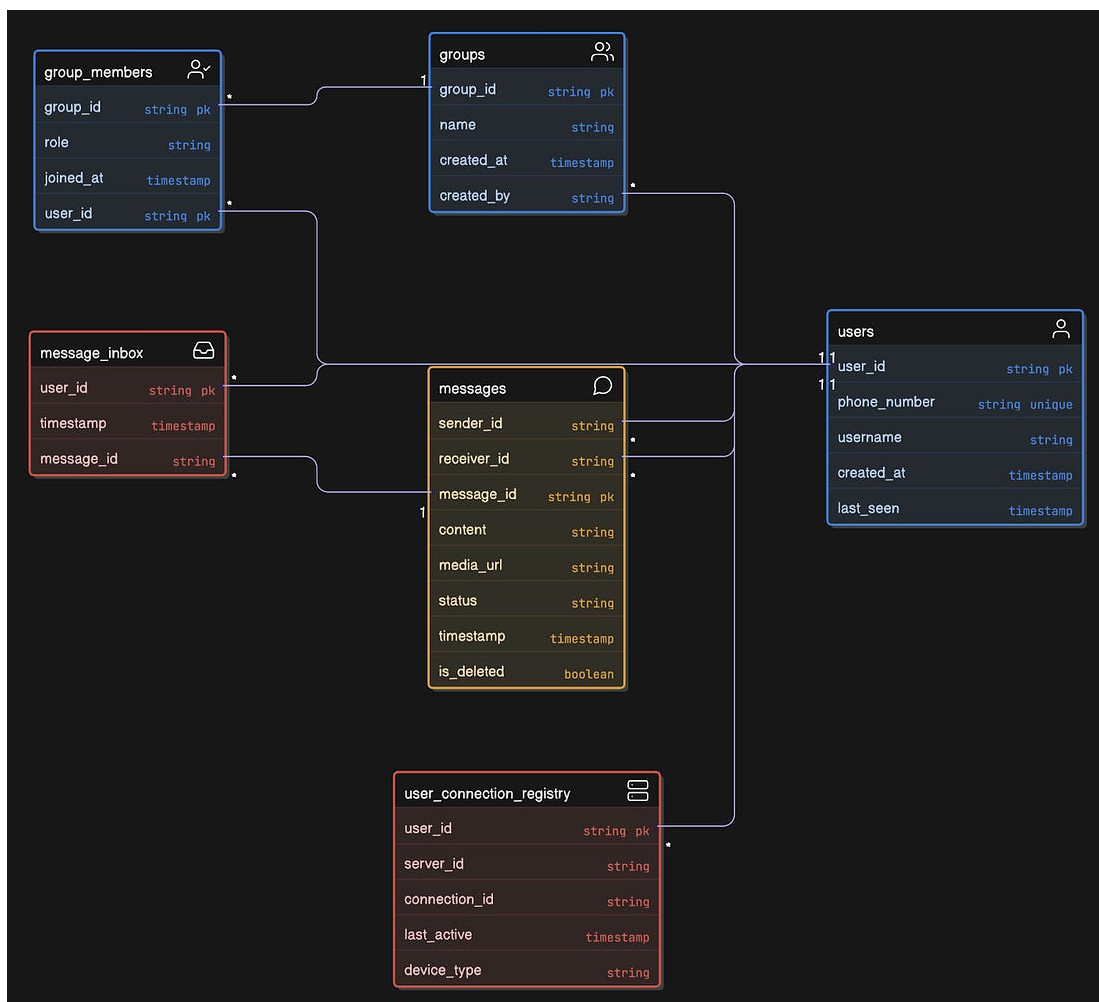
Tradeoffs:

Service discovery adds another system to maintain (Consul cluster needs to be highly available).

But the alternative is manual instance management, which doesn't scale and causes outages when you forget to update configurations.

Data Models

Let's design tables that actually support our use cases:



Users Table (SQL - PostgreSQL works)

```
Users

CREATE TABLE Users (
  user_id BIGINT PRIMARY KEY,
  phone_number VARCHAR(20) UNIQUE NOT NULL,
  username VARCHAR(50),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen TIMESTAMPTZ
);
```

Messages Table (NoSQL - Cassandra)

```
Messages

{
  "message_id": "20241105123456789001", // Timestamp-based ID
  "sender_id": "user123",
  "receiver_id": "user456", // Or group_id for groups
  "content": "The actual message text",
  "media_url": "https://cdn.example.com/file123.jpg", // Optional
  "status": "delivered", // sent, delivered, read
  "timestamp": 1699189496789,
  "is_deleted": false
}
```

Groups Table (SQL)

Groups & GroupMembers

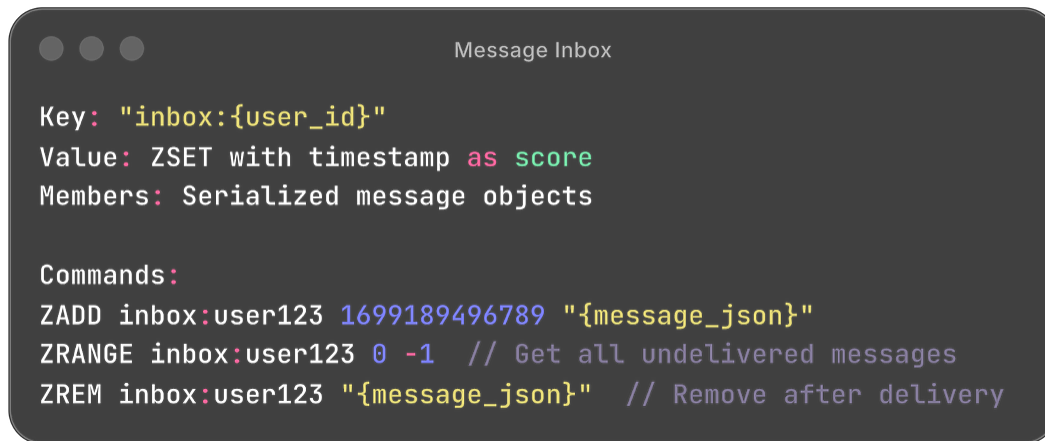
```
CREATE TABLE Groups (  
  group_id BIGINT PRIMARY KEY,  
  name VARCHAR(100),  
  created_by BIGINT REFERENCES Users(user_id),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE GroupMembers (  
  group_id BIGINT REFERENCES Groups(group_id),  
  user_id BIGINT REFERENCES Users(user_id),  
  role VARCHAR(20), // admin, member  
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  PRIMARY KEY (group_id, user_id)  
);
```

User Connection Registry (Redis)

User connection Redis

```
Key: "user:{user_id}"  
Value: {  
  "server_id": "chat-server-5",  
  "connection_id": "conn-xyz789",  
  "last_active": 1699189496789,  
  "device_type": "ios"  
}  
TTL: 300 seconds (refresh with each heartbeat)
```

Message Inbox (Redis)



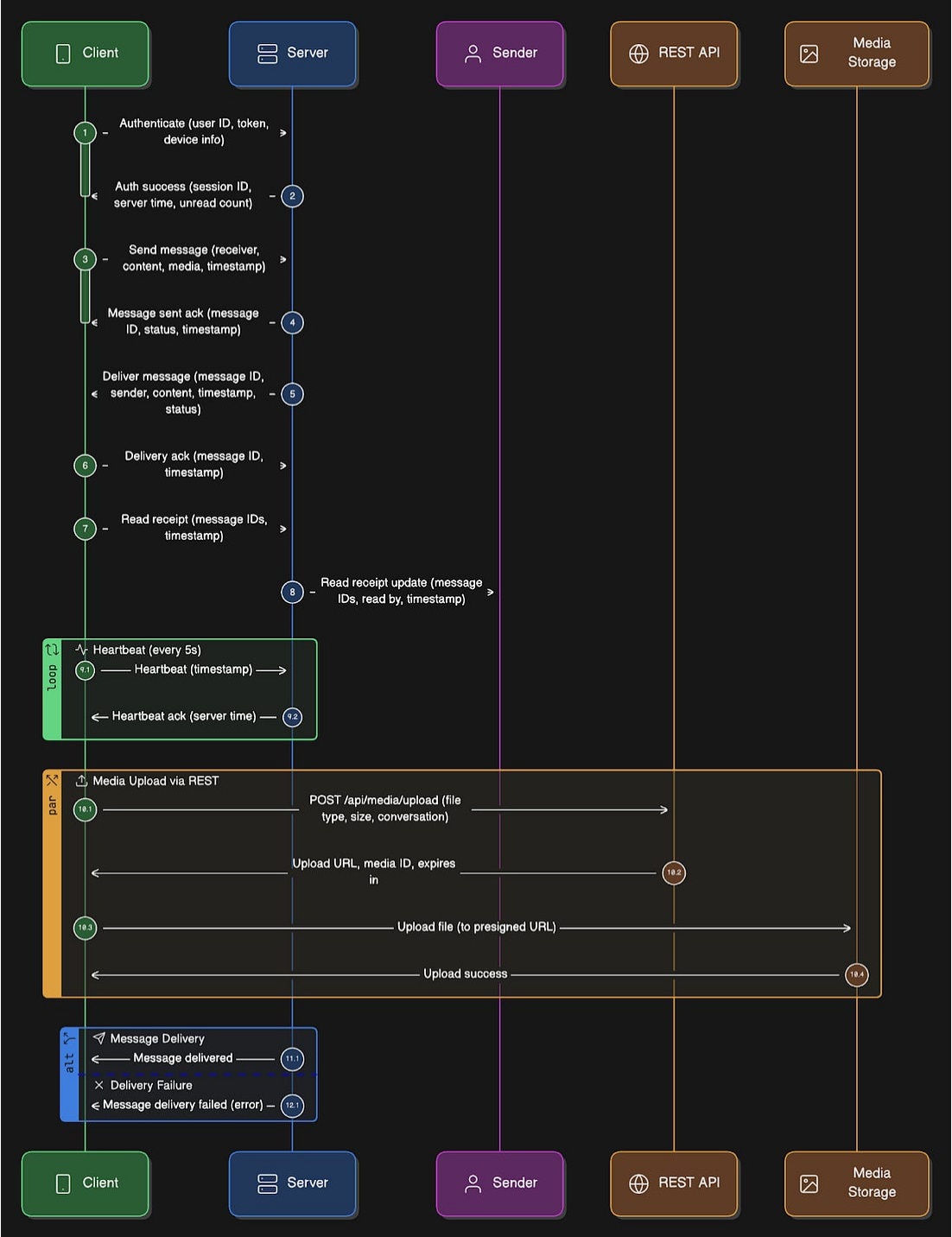
```
Message Inbox

Key: "inbox:{user_id}"
Value: ZSET with timestamp as score
Members: Serialized message objects

Commands:
ZADD inbox:user123 1699189496789 "{message_json}"
ZRANGE inbox:user123 0 -1 // Get all undelivered messages
ZREM inbox:user123 "{message_json}" // Remove after delivery
```

WebSocket and REST API Schemas

Now let's get concrete with actual message formats:



WebSocket Connection Establishment:

```
Client → Server (Initial handshake over WSS):  
{
```

```
"type": "auth",
"payload": {
  "user_id": "123",
  "auth_token": "jwt_token_here",
  "device_id": "phone-abc",
  "platform": "ios",
  "app_version": "2.24.5"
}
}
```

Server → Client (Auth success):

```
{
  "type": "auth_success",
  "payload": {
    "session_id": "sess_xyz789",
    "server_time": 1699189200000,
    "unread_count": 47
  }
}
```

Sending a Message:

Client → Server:

```
{
  "type": "message",
  "client_message_id": "client_abc123",
  "payload": {
    "receiver_id": "456",
    "content": "Hey, are you free tomorrow?",
    "media_url": null,
    "reply_to": null,
    "timestamp": 1699189200000
  }
}
```

Server → Client (ACK):

```
{
  "type": "message_ack",
  "client_message_id": "client_abc123",
  "payload": {
    "message_id": "20241105120000000001",
    "status": "sent",
    "timestamp": 1699189200123
  }
}
```

Receiving a Message:

Server → Client:

```
{
  "type": "message",
  "payload": {
    "message_id": "20241105120000000001",
    "sender_id": "456",
    "receiver_id": "123",
    "content": "Yes, what time works for you?",
    "media_url": null,
    "timestamp": 1699189200123,
    "status": "delivered"
  }
}
```

Client → Server (Delivery ACK):

```
{
  "type": "delivery_ack",
  "message_id": "20241105120000000001",
  "timestamp": 1699189200456
}
```

Read Receipt:

Client → Server (User opened chat):

```
{
  "type": "read_receipt",
  "message_ids": [
    "202411051200000000001",
    "202411051200000000002",
    "202411051200000000003"
  ],
  "timestamp": 1699189260000
}
```

Server → Original Sender:

```
{
  "type": "read_receipt",
  "payload": {
    "message_ids": ["202411051200000000001", ...],
    "read_by": "123",
    "timestamp": 1699189260000
  }
}
```

Presence Heartbeat:

Client → Server (every 5 seconds):

```
{
  "type": "heartbeat",
  "timestamp": 1699189200000
}
```

Server → Client:

```
{  
  "type": "heartbeat_ack",  
  "server_time": 1699189200123  
}
```

Media Upload Request:

Client → Server (REST API):

POST /api/media/upload

```
{  
  "file_type": "image/jpeg",  
  "file_size": 2457600,  
  "conversation_id": "conv_123_456"  
}
```

Server → Client:

```
{  
  "upload_url": "https://s3.amazonaws.com/bucket/signed_url...",  
  "media_id": "media_xyz789",  
  "expires_in": 3600  
}
```

Group Message:

Client → Server:

```
{  
  "type": "group_message",  
  "client_message_id": "client_def456",  
  "payload": {  
    "group_id": "group_789",  
    "content": "Meeting at 3 PM",  
    "media_url": null,  
    "timestamp": 1699189200000  
  }  
}
```



```
}  
}
```

Server fans out to all members, each receives:

```
{  
  "type": "group_message",  
  "payload": {  
    "message_id": "20241105120000000005",  
    "group_id": "group_789",  
    "sender_id": "123",  
    "content": "Meeting at 3 PM",  
    "timestamp": 1699189200123  
  }  
}
```

Ready for the best part?

The Online/Offline Control with Presence Server

We mentioned storing presence in Redis earlier, but let's dig into the architecture because presence is trickier than it looks...

The Dedicated Presence Service

While chat servers could handle presence, separating it into a dedicated service makes sense at scale.

Architecture:

- Presence Servers: Receive heartbeats, update status
- Redis Cluster: Stores current status for all users
- Presence Event Stream (Kafka): Publishes status changes

- Fan-out Service: Distributes status updates to interested users

How It Works (Heartbeat Flow):

Every 5 seconds while the user is active...

1. Client sends a heartbeat to the Chat Server
2. Chat Server forwards to the Presence Server
3. Presence Server updates Redis:

A terminal window with a dark background and rounded corners. It has three window control buttons (red, yellow, green) in the top-left corner. The title bar text is "15 second expiry". The command being executed is a Redis SET command with an expiration. The command is: SET user:{user_id}:presence { status: "online", last_seen: timestamp, device: "ios", server_id: "chat-5" } EX 15. The command is color-coded: SET is yellow, user:{user_id}:presence is blue, { is blue, status: "online", last_seen: timestamp, device: "ios", server_id: "chat-5" is blue, } is blue, EX is yellow, and 15 is blue.

```
15 second expiry

SET user:{user_id}:presence {
  status: "online",
  last_seen: timestamp,
  device: "ios",
  server_id: "chat-5"
} EX 15
```

The 15-second TTL is intentional.

If heartbeats stop (network dies or the app crashes), the user automatically goes offline after 15 seconds.

Status Change Detection (Presence Server logic):

1. Check the previous status in Redis
2. If changed (online → offline or vice versa):
 - Publish to Kafka: {user_id: "123", status: "offline", timestamp: ...}
3. Fan-out Service consumes from Kafka

4. Queries *“who needs to know about user 123’s status?”*

- Friends list
- Recent chat partners
- Group members they share

5. Sends the status update to those users via their chat servers

The Subscription Problem:

If User A has 500 contacts, should they get notified every time any of their contacts come online or go offline?

That’s too much traffic!

Optimization - Smart Subscriptions:

User A only subscribes to presence updates for:

- Active chats (opened in the last 7 days)
- Pinned contacts
- Ongoing group chats

This reduces presence traffic by 80-90% while still feeling real-time where it matters.

Last Seen Timestamp:

When User B checks User A’s profile, they see “last seen 2 hours ago”. This comes from the `last_seen` field in Redis.

When we display it:

If `last_seen < 15 seconds ago`: “online”

If last_seen < 5 minutes: "last seen just now"

If last_seen < 1 hour: "last seen 23 minutes ago"

If last_seen < 24 hours: "last seen at 3:45 PM"

Else: "last seen Oct 21"

Privacy Controls:

Users can hide their status. This is just a flag in the user profile.

When User A requests User B's status:

1. Check User B's privacy settings
2. If B allows all: return status
3. If B allows contacts only: check if A is in B's contacts
4. If B allows nobody: return *"last seen a long time ago."*

Tradeoffs:

Running a separate presence service adds operational complexity (another thing to monitor and scale).

But mixing presence with chat traffic creates scaling headaches. Chat traffic is spiky and unpredictable. Presence traffic is steady and predictable. They have different scaling characteristics, so separating them makes sense.

Multi-Device Sync

Modern users expect their messages to be available on phones, tablets, and desktops.

This is harder than it sounds...

The Problem:

User A sends a message from their phone.

User A's tablet needs to see it in the conversation. User B receives the message on all their devices. How do we keep everything in sync without exploding the complexity?

Architecture:

Each device gets its own connection to the chat server. When User A has 3 devices online:

Redis Connection Cache:

user:123:devices ⇒ [

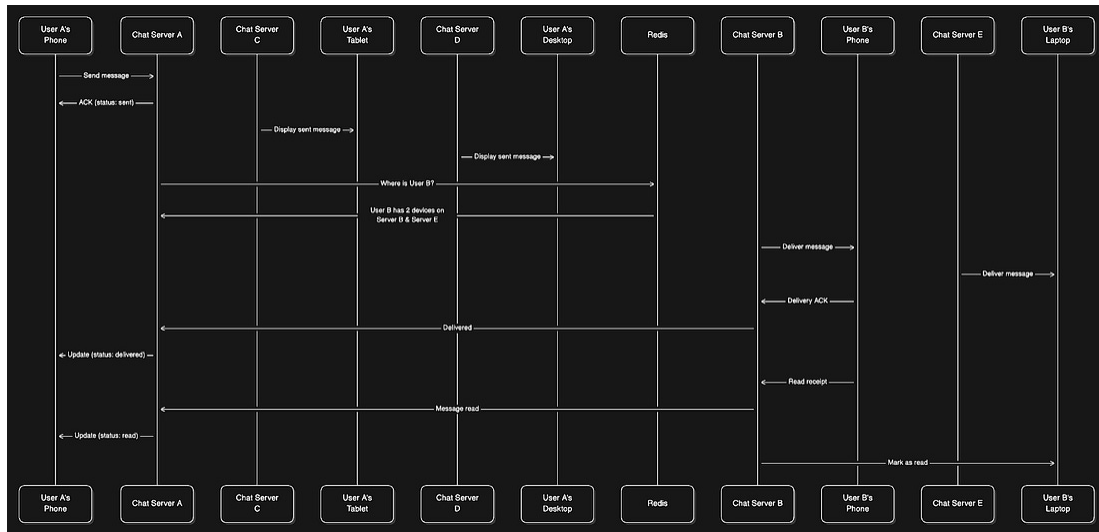
{device_id: "phone-abc", server_id: "chat-1", platform: "ios"},

{device_id: "tablet-def", server_id: "chat-2", platform: "ios"},

{device_id: "desktop-ghi", server_id: "chat-3", platform: "web"}

]

Message Flow:



Sending (User A's phone sends message):

1. Phone sends a message to Chat Server A
2. Server A process the message, generates a message_id
3. Server A delivers to User B (all their devices)
4. Server A also delivers to User A's OTHER devices:
 - Publishes to Redis: channel:user:123:sent
 - Tablet and Desktop receive via their servers
 - They display the message in the conversation

Receiving (User B gets message on all devices):

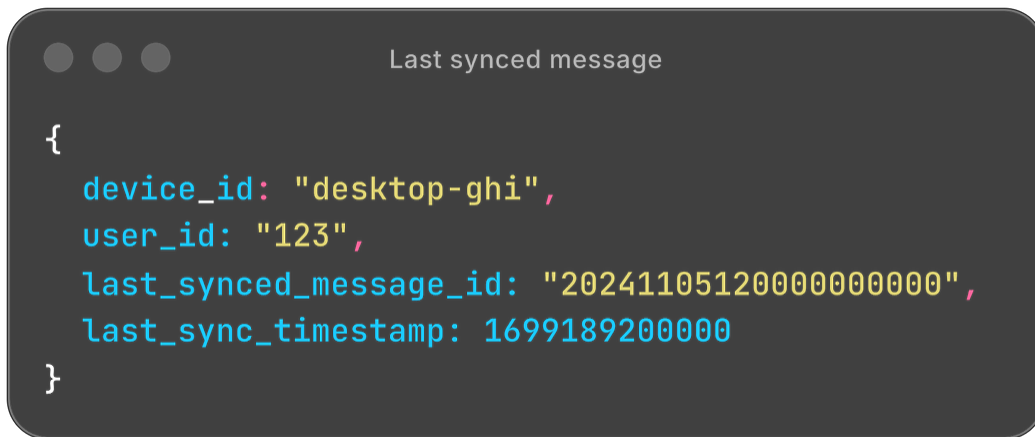
1. Message arrives for User B
2. Server checks Redis for all B's active devices
3. Routes messages to each device's chat server
4. Each device receives the message
5. Whichever device reads first sends a read receipt
6. Read receipt delivered to all of B's other devices (marks as read everywhere)

Sync State Problem:

User opens the app on desktop after being offline. Their phone has been receiving messages. So the desktop needs to catch up.

Solution - Sync Point:

Each device tracks its last synced message ID:



```
{
  device_id: "desktop-ghi",
  user_id: "123",
  last_synced_message_id: "20241105120000000000",
  last_sync_timestamp: 1699189200000
}
```

When desktop reconnects:

1. Sends sync request: *"I have messages up to X"*
2. Server queries database:
 - *"Give me all messages for user 123 after message_id X"*
3. Server sends a batch of missed messages
4. Desktop processes and updates last_synced_message_id

Message Send Failure and Client-Side Queue:

What happens if a user sends a message but their network fails before it reaches the server?

Client-Side Solution:

```
Message send failure

const pendingMessages = [];

async function sendMessage(content, receiverId) {
  const clientMsg = {
    client_msg_id: generateUUID(),
    content,
    receiver_id: receiverId,
    timestamp: Date.now(),
    status: 'pending'
  };

  await localDB.save(clientMsg);
  pendingMessages.push(clientMsg);
  displayMessage(clientMsg, { status: 'sending' });

  try {
    const ack = await websocket.send(clientMsg);
    clientMsg.status = 'sent';
    clientMsg.message_id = ack.message_id;
    await localDB.update(clientMsg);
    displayMessage(clientMsg, { status: 'sent' });
  } catch (networkError) {
    clientMsg.status = 'failed';
    await localDB.update(clientMsg);
    displayMessage(clientMsg, { status: 'failed' });
  }
}

setInterval(async () => {
  const pending = await localDB.getPendingMessages();
  for (const msg of pending) {
    if (!websocket.connected) continue;
    try {
      await websocket.send(msg);
      await localDB.delete(msg.client_msg_id);
    } catch (error) {
      // retry next interval
    }
  }
}, 5000);
```

The app maintains a local queue of outbound messages:

1. User sees their message immediately in the chat (optimistic UI)
2. Message persists locally even if the app crashes
3. Automatic retry when the network returns

4. Server deduplication (using `client_msg_id`) prevents duplicates
5. User can manually retry failed messages

Conflicts - Which Device is the Source of Truth?

User A deletes a message on their phone. But the tablet is offline.

So what happens when the tablet comes back online?

Conflict Resolution:

Every mutation (delete, edit, etc.) gets a version number:



```
{
  message_id: "20241105120000000000",
  content: "Hello",
  version: 3,
  operations: [
    {op: "create", timestamp: 1699189200000, device: "phone"},
    {op: "edit", timestamp: 1699189260000, device: "phone", new_content: "Hi"},
    {op: "delete", timestamp: 1699189320000, device: "phone"}
  ]
}
```

When tablet syncs:

1. Receives version 3 with delete operation
2. Checks local version (still at version 1)
3. Applies operations 2 and 3
4. Message deleted on tablet too

Last-write-wins with version numbers prevents conflicts. The server maintains the authoritative state.

Optimization - Incremental Sync:

Sending the entire message history to a new device is expensive.
Instead:

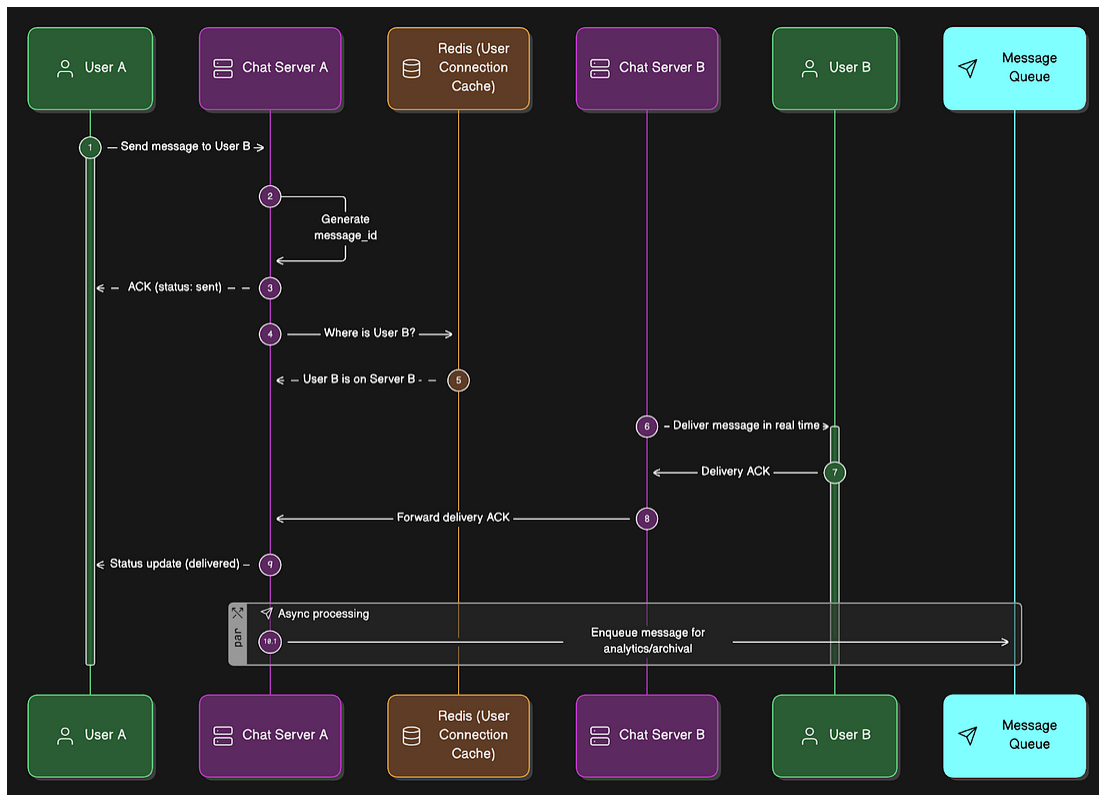
1. Device requests messages for the last 30 days (most users only need recent history)
2. Older messages are fetched on demand when the user scrolls up
3. Older messages are archived in cold storage and loaded lazily

Tradeoffs:

Multi-device support roughly doubles the delivery workload (each message goes to multiple devices per user). But users demand it, so we optimize by batching updates and being smart about sync windows.

The Core Flows on How Messages Actually Move

Sending a Message (Both Users Online)



1. User A types a message, hits send
2. Message goes through WebSocket to Chat Server A
3. Server A generates a unique message ID (timestamp-based: 20241105123456789001)
4. Server A immediately sends ACK back to User A (message shows as "sent")
5. Server A pushes message to Message Queue for persistence
6. Server A checks User Connection Cache: "Where is User B?"
7. Cache returns: "User B is on Chat Server B"
8. Server A routes message to Server B via Redis Pub/Sub
9. Server B pushes message through User B's WebSocket connection
10. User B's app receives message, sends delivery ACK back to Server B
11. Server B forwards delivery ACK to Server A

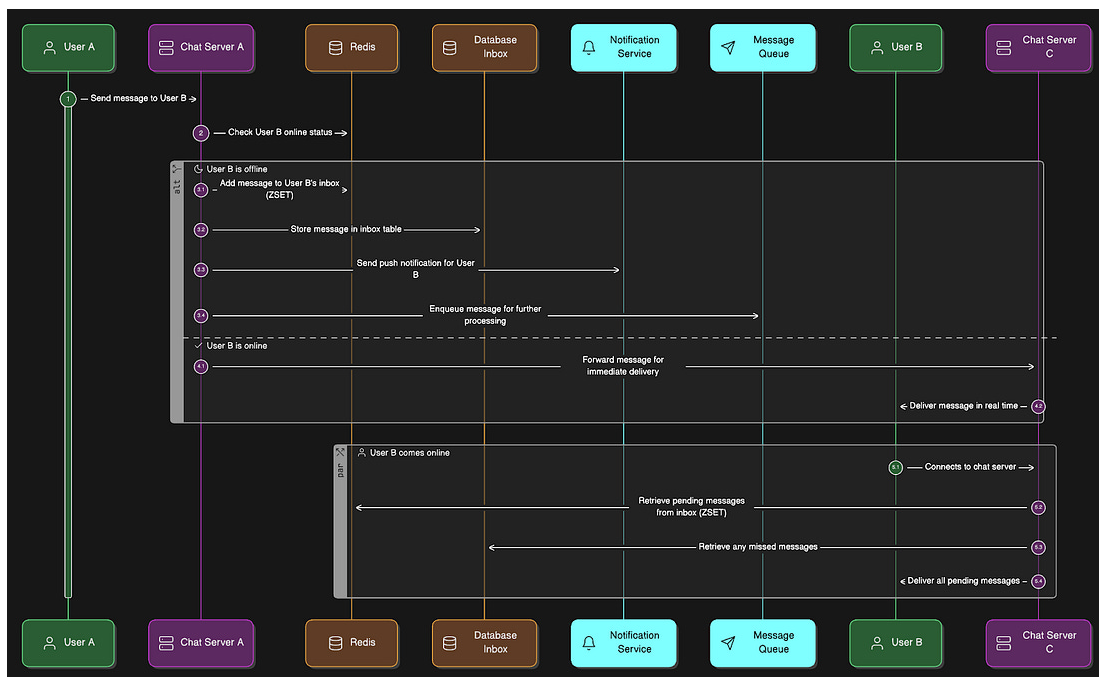
12. Server A sends delivery status update to User A (message shows as “delivered”)
13. When User B opens the chat, their app sends read ACK
14. Read status flows back to User A (message shows as “read”)

Meanwhile, the Message Queue consumer writes the message to the Message Database for history.

Sending a Message (User B Offline)

Steps 1-5 same as above.

Then:



6. Server A checks User Connection Cache: “Where is User B?”
7. Cache returns: “User B is offline”
8. Server A writes message to User B’s inbox in Redis (sorted set by timestamp)
9. Server A also writes to database inbox table (dual write for safety)

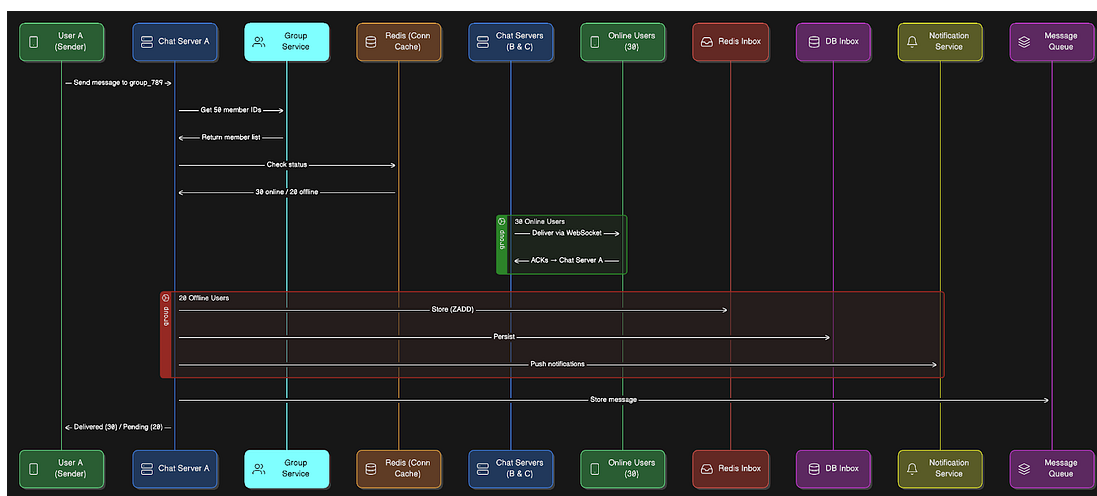
10. Server A sends message to Notification Service
11. Notification Service triggers push notification through APNs/FCM
12. Message Queue consumer writes to Message Database

When User B comes back online:

1. User B's app opens, establishes a WebSocket connection with Chat Server C
2. Server C updates User Connection Cache
3. Server C checks Redis inbox for User B
4. Finds undelivered messages, sends them through WebSocket
5. User B's app sends delivery ACKs for each message
6. Server C removes messages from Redis inbox
7. Server C marks messages as delivered in database inbox
8. Server C forwards delivery status updates to original senders

Group Messages: The Fan-Out Problem

User A sends a message to a 50-person group:



1. Message arrives at Chat Server A through User A's WebSocket

2. Server A queries Group Service: “Who’s in this group?”
3. Gets back 50 user IDs
4. Server A checks the User Connection Cache for all 50 users
5. Finds 30 online, 20 offline
6. For each online user:
 - Looks up which chat server they’re on
 - Routes message to that server via Redis Pub/Sub
 - That server delivers via WebSocket
7. For each offline user:
 - Writes to their Redis inbox
 - Writes to database inbox
 - Sends to the Notification Service for push notification
8. Server A pushes message to Message Queue (one write for the entire group)
9. Each recipient’s delivery ACK flows back to Server A
10. Server A aggregates read receipts for the group

This is why groups have size limits.

A 1000-person group means 1000 delivery operations per message. The fan-out workload grows linearly with group size. WhatsApp limits groups to 1024 members. Signal does 1000.

Telegram does more, but with different trade-offs.

Handling Media Files Without Routing Them Through Chat Servers

The wrong way is obvious:

client uploads image to chat server, chat server stores in database,
recipient downloads from chat server.

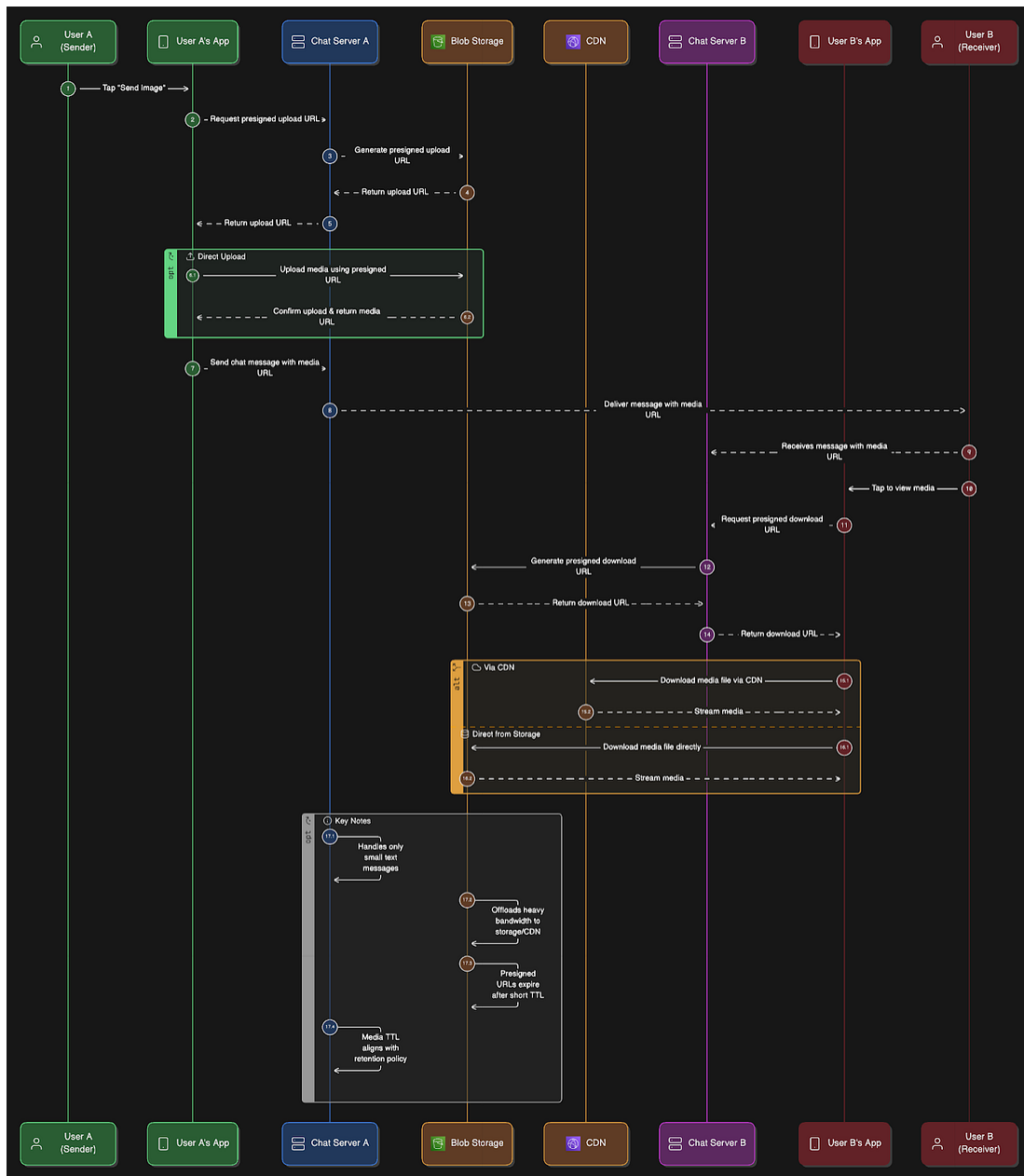
This is not scalable.

Your chat servers are now bandwidth-limited by file transfers. They're not doing what they're good at (routing messages quickly), they're doing what they're terrible at (moving large blobs around).

The right approach uses pre-signed URLs

A presigned URL is a temporary URL that grants access to upload or download a file from storage without requiring credentials.

Think of it like a visitor badge that expires after a few hours.



Upload flow:

1. User A wants to send an image
2. App requests upload URL from Chat Server A
3. Server A generates a pre-signed URL for blob storage (AWS S3, Google Cloud Storage, etc.)
4. Server A returns a pre-signed URL to User A

5. User A's app uploads directly to blob storage using that URL
6. Blob storage returns the URL of the uploaded file
7. User A's app sends a message with a media URL through the normal message flow
8. Message containing URL gets delivered to recipients

Download flow:

1. User B receives message with media URL
2. When User B clicks to view, app requests download URL
3. Chat Server B generates presigned URL for blob storage
4. User B's app downloads directly from blob storage or CDN

Why we chose this:

Chat servers only route small messages with URLs.

Blob storage and CDN handle the heavy bandwidth. Your chat servers stay fast and focused on their job. Media files have a TTL (Time To Live, automatic expiration) of 30 days, matching message retention, and are automatically deleted.

Tradeoffs:

Slightly more complex flow (extra request to get presigned URL), but the performance and cost benefits are massive.

Without this, a single chat server handling 100,000 connections can get crushed by just 100 users simultaneously uploading 10MB videos.

We'll cover scalability and much more in Part 2!



I'd like to thank [Hayk](#) for writing this newsletter!

- Also don't forget to join his **YouTube channel**.

It'll help you master the essential system design skills, not just in theory but in practice, and land senior developer roles.



👉 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Thank you for supporting this newsletter.

You are now 190,001+ readers strong, very close to 191k. Let's try to get 191k readers by 21 December. Consider sharing this post with your friends and get rewards.

Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share